

Break down the code for contract signing date into the corresponding match list date

To see exactly how the **Contract Signing Date (Start Date)** transforms and links up with the **Salary Payment Date** inside your `ContractList`, we can break the logic down into two clear steps.

Here is the exact step-by-step breakdown of what the code is doing under the hood, along with visual callouts showing where to look.

Step 1: The Contract Date Self-Check (`Contract.java`)

Before a contract can even be added to your list, the code checks if the **Signing Date** (`startDate`) actually happens *before* or on the **End Date** (`endDate`).

Look at this specific block inside your `Contract.java` :

Java



```
// Line to find: inside validateContractDates()
int startYear = Integer.parseInt(startDate.substring(0, 4));
int startMonth = Integer.parseInt(startDate.substring(5, 7));

int endYear = Integer.parseInt(endDate.substring(0, 4));
int endMonth = Integer.parseInt(endDate.substring(5, 7));
```

What you are looking at:

If your contract string input is "2026-01-15", Java counts characters starting at index position 0.

- `substring(0, 4)` slices out the first 4 characters ("2026").
- `substring(5, 7)` skips the dash and slices out the next 2 characters ("01").
- `Integer.parseInt()` changes those words into pure numbers: 2026 and 1.

The Sequence Match Logic:

Java



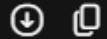
```
if (startYear < endYear) {  
    return true; // Example: 2026 is less than 2027. Perfect!  
} else if (startYear == endYear) {  
    return startMonth <= endMonth; // Same year! Check if Start Month (01) <= End Mo  
}
```

Step 2: Correlating with the Salary Date (`ContractList.java`)

Once a contract is saved, you try to add a monthly salary receipt. The system needs to ensure the paycheck date matches the contract terms.

Look at this specific tracking block inside `ContractList.java` :

Java



```
// Line to find: inside addSalary()  
int contractStartValue = (startYear * 12) + startMonth;  
int contractEndValue = (endYear * 12) + endMonth;  
int salaryValue = (newSalary.getYear() * 12) + newSalary.getMonth();
```

What you are looking at:

Because comparing months across different years with pure `if` statements creates complex, jumbled code, we use a math trick to turn dates into a single **Timeline Value** (total months elapsed).

Let's trace a real example using **Contract:** 2026-01-01 to 2026-06-30 and **Salary Payment:** Month 3, Year 2026 .

- `contractStartValue` calculation: $(2026 \times 12) + 1 = 24313$
- `contractEndValue` calculation: $(2026 \times 12) + 6 = 24318$
- `salaryValue` calculation: $(2026 \times 12) + 3 = 24315$

Step 3: The Final Intersection Boundary Check

Now that the dates are flat numbers, the code performs a simple boundary check to confirm the salary fits the timeline list:

Step 3: The Final Intersection Boundary Check

Now that the dates are flat numbers, the code performs a simple boundary check to confirm the salary fits the timeline list:

Java



```
// Line to find: the final IF condition in ContractList.java
if (salaryValue >= contractStartValue && salaryValue <= contractEndValue) {
    listAllSalaries.add(newSalary); // The salary matches! Save it.
}
```

Visualizing the Boundary Result:

Plaintext



TIMELINE SCALE:	24313	24315	24318
	[Start]	[Salary]	[End]
	Jan 2026	Mar 2026	June 2026
	(Valid)	(MATCH)	(Valid)

Because 24315 sits firmly between 24313 and 24318, the expression evaluates to `true`. The code confirms the payment matches the active contract timeline, bypasses any out-of-bounds error blocks, and saves the entry cleanly.